Software Requirements Specification (SRS)

for

Campus Event & Club Management Application

Prepared for:

Md Abu Sayed

Independent University, Bangladesh (IUB)

Course No: CSE 309

CourseTitle: Web Applications & Internet (Section 03)

Prepared by:

Name	ID
Nishe Akther Nipa	2321153
S. M. Shoikat Azad Shawon	2320246
Ahnaf Pushon	2320401

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this application is to develop a centralized web-based platform for the students, clubs, and administration of Independent University, Bangladesh (IUB). The system aims to eliminate the fragmentation of campus event information currently spread across social media, emails, and messaging groups. It will serve as a single source of truth for discovering events, managing club memberships, and handling event registrations.

1.2 Document Conventions

- **Shall:** Indicates a mandatory requirement that must be implemented.
- **Should:** Indicates a recommended requirement that is desirable but not mandatory for the initial prototype.
- **May:** Indicates an optional requirement.

1.3 Intended Audience

This document is intended for:

- IUB Students, for event discovery, club memberships, and activity management.
- The course instructor (Md Abu Sayed) for project evaluation.
- The development team (Nipa, Shawon, Pushon) as a technical guide.
- Future maintainers or evaluators of the application.

1.4 Project Scope

The **Campus Event & Club Management Application** will be a full-stack web application.

- **In Scope:** Student authentication (via IUB email), a personalized dashboard, event browsing/searching, event registration (with waitlist support), club creation/membership management, admin event creation, and a notification system.

- **Out of Scope (Initial Prototype):** Official integration with IUB's primary student portal (SSO), mobile native apps (PWA/Web responsive only), and handling of financial transactions or payments.

2. Overall Description

2.1 Product Perspective

This system is a new, self-contained web product. It will operate independently but will utilize email services (SMTP) for notifications and verification. It consists of:

- **A React Frontend:** Hosted on Vercel, providing a responsive user interface.
- **A FastAPI Backend:** Hosted on Render/AWS, providing RESTful APIs.
- **A PostgreSQL Database:** Hosted on Supabase/RDS, storing all application data.

2.2 User Characteristics

The system defines three primary user roles:

1. **Student (General User):**
    - Characteristics: Registered IUB student.
    - Capabilities: Browse events, register for events, join clubs, view dashboard, manage profile.
2. **Club Admin / Executive:**
    - Characteristics: A student designated as the manager/executive of a specific club.
    - Capabilities: All student capabilities, plus create/manage events, approve/reject club membership requests, and send announcements.
3. **Super Admin (System/Student Affairs):**
    - Characteristics: Administrator designated by the university or system owner.
    - Capabilities: Oversee all clubs, delete inappropriate content/events, manage user disputes, and manage system-wide settings.

2.3 Operating Environment

- **Client-side:** Modern web browsers (Google Chrome, Mozilla Firefox, Safari) and mobile viewports (iOS/Android responsive design).
- **Server-side:** Linux-based cloud environment (Render/AWS) running Python (FastAPI).
- **Network:** Requires a stable internet connection with HTTPS protocol.

2.4 Design and Implementation Constraints

- **Authentication:** User registration is restricted to valid @iub.edu.bd email addresses.
- **Technology Mandate:** Must strictly use the tech stack: **React.js** (frontend), **FastAPI** (backend), **PostgreSQL** (database), and **GitHub** (version control).
- **Architecture:** The backend must follow a layered architecture: API Router → Service → Repository → Database Model/Schema.

2.5 Assumptions and Dependencies

- **Assumptions:** It is assumed that all students have access to their IUB-provided email inbox for OTP/password reset functionalities.
- **Dependencies:** The system relies heavily on the availability of the internet and the chosen cloud hosting platforms (Vercel, Render, Supabase).

3. Technology Stack Summary

Specific technologies:

Component	Technology	Justification
Frontend	React (Vite), React Router, Axios	Mandated for component-based UI and API integration.
Backend	FastAPI (Python)	Mandated for implementing layered

Component	Technology	Justification
		REST APIs.
Database	PostgreSQL with SQLAlchemy ORM	Mandated for database integration with CRUD.
Deployment	Vercel (FE), Render (BE), Supabase (DB)	Mandated for full-stack deployment.

4. Specific Requirements (Functional Requirements)

This section details the functional requirements of the Campus Event & Club Management Application, categorized by primary system features and user roles. Each requirement is assigned a unique identifier for traceability. All requirements in this section are classified as Must unless stated otherwise.

4.1 User Authentication and Profile Management

Requirement ID	Description
4.1.1	Student Registration: The system shall allow users to register for a new account exclusively using a valid @iub.edu.bd email address.
4.1.2	Login/Logout: The system shall allow registered users to log in using their email and password, and log out of their active session securely.
4.1.3	Role-Based Access Control (RBAC): The system shall assign and enforce user roles (Student, Club Admin, Super Admin). Access to specific pages and API endpoints must be restricted based on these roles.
4.1.4	Profile Management: The system shall allow users to view and update their profile information, including their full name, Student ID, and department (e.g., CSE, EEE, BBA).
4.1.5	Password Recovery: The system shall provide a "Forgot Password" mechanism that sends a secure reset link or OTP to the user's registered IUB email.

4.2 Event Management (Club Admins & Super Admins)

Requirement ID	Description
4.2.1	Event Creation: The system shall allow Club Admins to create a new event by providing: Event Title, Description, Date, Start/End Time, Venue (or online link), Maximum Capacity, and a Poster Image URL.

Requirement ID	Description
4.2.2	Event Modification: The system shall allow the creator of an event (or a Super Admin) to edit event details or cancel the event entirely.
4.2.3	Attendee Management: The system shall allow Club Admins to view a real-time roster (list) of all students registered for their specific events.
4.2.4	Attendance Tracking: The system may allow Club Admins to mark the attendance status of registered users during or after the event. (Priority: Could – optional for prototype)

4.3 Event Discovery and Registration (Students)

Requirement ID	Description
4.3.1	Event Feed: The system shall display a feed or calendar view of all upcoming approved campus events to authenticated users.
4.3.2	Search and Filter: The system shall allow users to search for events by keyword (title/description) and filter events by specific

Requirement ID	Description
	clubs or date ranges.
4.3.3	Event Registration: The system shall allow a student to register for an upcoming event with a single click, provided the event has not reached its maximum capacity.
4.3.4	Waitlist Functionality: If an event is at maximum capacity, the system should allow students to join a "Waitlist." If a registered student cancels, the first waitlisted student is automatically moved to the registered list.
4.3.5	Registration Cancellation: The system shall allow students to cancel their registration for an event before the event's start time.

4.4 Club Membership Management

Requirement ID	Description
4.4.1	Club Directory: The system shall provide a directory listing all registered campus clubs along with their descriptions and current executive panels.
4.4.2	Membership Application: The system shall allow general students

Requirement ID	Description
	to submit a request to join a specific club.
4.4.3	Application Review: The system shall allow Club Admins to view pending membership requests and securely Approve or Reject them.
4.4.4	Member Roster: The system shall allow Club Admins to view and manage (e.g., remove) current members of their club.

4.5 Personalized Dashboard

Requirement ID	Description
4.5.1	Student Dashboard: The system shall provide a personalized dashboard for students displaying their upcoming registered events, waitlisted events, and current club memberships.
4.5.2	Admin Dashboard: The system shall provide an admin dashboard for Club Admins displaying quick statistics (total members, upcoming club events, pending membership requests).

4.6 Notification System

Requirement ID	Description
4.6.1	Registration Confirmation: The system shall generate an in-app (or email) notification to a student upon successful event registration.

Requirement ID	Description
4.6.2	Event Updates: The system shall automatically notify all registered attendees if an event's time, venue is changed, or if the event is cancelled.
4.6.3	Membership Status: The system should notify students when their club membership application is approved or rejected.

5. External Interfaces

5.1 User InterfacesThe application shall feature a responsive web interface optimized for both desktop browsers and mobile viewports.

- The frontend shall be built using **React** and styled appropriately to ensure a modern, accessible user
- experience.
- Error messages, success toasts, and form validation feedback shall be clearly displayed to guide the user.

5.2 Software Interfaces

- **Database Interface:** The backend (FastAPI) shall communicate with the **PostgreSQL** database using SQLAlchemy ORM to perform all CRUD operations.
- **Authentication Interface:** The system will internally handle JWT (JSON Web Tokens) for session management and API endpoint protection.
- **Email Gateway (SMTP):** The backend should interface with an external email service to handle outbound notification emails and OTPs.

5.3 Hardware Interfaces

- As a web application, there are no direct hardware interfaces required beyond the standard client-server network interaction over TCP/IP (HTTP/HTTPS).

5.4 Communication Interfaces

- All communication between the React frontend and the FastAPI backend shall occur via

RESTful API calls over HTTPS.

- Data payloads will strictly use JSON format.

6. Non-Functional Requirements

6.1 Performance

- The event feed shall load within **3 seconds** for up to **100 concurrent users** in the prototype environment.

6.2 Security

- All passwords shall be hashed using **encryption** before storage.
- All API endpoints except /login and /register shall require a valid **JWT** token.

6.3 Usability

- A student shall be able to complete event registration in **at most 3 clicks** from the event list.

6.4 Reliability

- The system shall maintain **99% uptime** during normal university hours (8 AM – 10 PM).

6.5 Maintainability

- The backend shall strictly follow a **layered architecture** (Router → Service → Repository → Model) as mandated by the course.

6.6 Scalability (for future)

- The database schema shall support at least **1,000 users** and **500 events** without major redesign.

7. Appendix / Database Schema

Appendix A – Preliminary Database Schema

Table	Key Fields
users	id, email, password_hash, role (student/club_admin/super_admin), name, student_id, department
clubs	id, name, description, admin_user_id (FK)
events	id, title, description, date, start_time, end_time, venue, capacity, poster_url, status (draft/published/cancelled), club_id (FK)
registrations	id, user_id (FK), event_id (FK), status (registered/waitlisted), registered_at
memberships	id, user_id (FK), club_id (FK), status (pending/approved/rejected), applied_at
notifications	id, user_id (FK), type, message, is_read, created_at